

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №1
по курсу «Технологии Параллельного Программирования»

Обработка изображений на GPU. Фильтры.

Выполнил: О.С. Концебалов

Группа: М8О-409Б-22

Преподаватели: А.Ю. Морозов,

Е.Е. Заяц

Москва, 2025

Условие

Цель работы: работы. Научиться использовать GPU для обработки изображений. Использование текстурной памяти и двумерной сетки потоков.

Формат изображений. Изображение является бинарным файлом, со следующей структурой:

width(w)	height(h)	r	g	b	a	r	g	b	a	r	g	b	a	...	r	g	b	a	r	g	b	a
4 байта, int	4 байта, int	4 байта, значение пикселя [1,1]				4 байта, значение пикселя [2,1]				4 байта, значение пикселя [3,1]				...	4 байта, значение пикселя [w - 1, h]				4 байта значение пикселя [w,h]			

В первых восьми байтах записывается размер изображения, далее построчно все значения пикселей, где

- r -- красная составляющая цвета пикселя
- g -- зеленая составляющая цвета пикселя
- b -- синяя составляющая цвета пикселя
- a -- значение альфа-канала пикселя

Пример картинки размером 2 на 2, синего цвета, в шестнадцатеричной записи:

02000000 02000000 0000FF00 0000FF00 0000FF00 0000FF00

Студентам предлагается самостоятельно написать конвертер на *любом* языке программирования для работы с вышеописанным форматом.

В данной лабораторной работе используются только цветовые составляющие изображения (r g b), альфа-канал не учитывается. При расчетах значений допускается ошибка в ± 1 . Ограничение: $w < 2^{16}$ и $h < 2^{16}$. Во всех вариантах, кроме 2-го и 4-го, в пограничном случае, необходимо "расширять" изображение за его границы, при этом значения соответствующих пикселей дублируют граничные. То есть, для любых индексов i и j, координаты пикселя [ip, jp] будут определяться следующим образом:

ip := max(min(i, h), 1)

jp := max(min(j, w), 1)

Вариант 7. Выделение контуров. Метод Собеля.

Входные данные. На первой строке задается путь к исходному изображению, на второй, путь к конечному изображению. $w \cdot h \leq 10^8$.

Пример:

Входной файл	hex: in.data	hex: out.data
in.data out.data	03000000 03000000 01020300 04050600 07080900 09080700 06050400 03020100 00000000 14141400 00000000	03000000 03000000 14141400 0C0C0C00 11111100 13131300 15151500 18181800 39393900 14141400 3F3F3F00
in.data out.data	03000000 03000000 00000000 00000000 00000000 00000000 80808000 00000000 00000000 00000000 00000000	03000000 03000000 B5B5B500 FFFFFFFF00 B5B5B500 FFFFFFF00 00000000 FFFFFFFF00 B5B5B500 FFFFFFFF00 B5B5B500

Программное и аппаратное обеспечение

OS

Семейство	Windows
Версия	10.0.19045
Релиз	10
Платформа	Windows-10-10.0.19045
Архитектура	AMD64

CPU

Модель	AMD Ryzen 5 5600H with Radeon Graphics
Архитектура	AMD64
Разрядность	64
Частота (паспортная)	3.2940 GHz
Частота (факт.)	3.3010 GHz
Ядер (физ.)	6
Потоков (лог.)	12

RAM

Всего	15.35 GB
Доступно	4.35 GB
Swap	6.00 GB

Диски

Диск	ФС	Всего, GB	Использовано, GB	Свободно, GB	Использовано, %
C:\	NTFS	470.94	391.13	79.82	83.05
D:\	FAT32	0.50	0.00	0.50	0.00

GPU

Compute capability	8.6
Name	NVIDIA GeForce RTX 3050 Ti Laptop GPU
Total Global Memory	4294443008
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)
Total constant memory	65536
Multiprocessors count	20

Используемая IDE: Visual Studio Code

Версия компилятора:

```
C:\Program Files\Microsoft Visual Studio\2022\Community>nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed Aug 20 13:58:20 Pacific Daylight Time 2025
Cuda compilation tools, release 13.0, V13.0.88
Build cuda_13.0.r13.0/compiler.36424714_0
```

Метод решения

В программе данные считываются из бинарного файла формата int32 w, int32 h, RGBA... в вектор `std::vector<Pixel>`. Далее на GPU под вход создаётся `cudaArray` и в неё построчно копируется изображение. К этой памяти привязывается `texture object` (`cudaTextureObject_t`) с параметрами: `addressMode=Clamp` по X/Y (аппаратное «расширение» границ), `filterMode=Point` (без интерполяции), `normalizedCoords=0` (обращение в пиксельных координатах), `readMode=ElementType` (чтение как `uchar4`). Под выход выделяется линейный буфер `d_out`.

Запуск выполняется двумерной сеткой `grid(64,64)` и блоками `block(32,8)`. Каждый поток начинает с координат (`xStart`, `yStart`) и обходит кадр с шагами `xStep = blockDim.x * gridDim.x`, `yStep = blockDim.y * gridDim.y`, что масштабируется на любые размеры `w×h`. Внутри ядра соседние пиксели окна `3×3` читаются из текстуры вызовом `tex2D<uchar4>(texIn, x+dx+0.5f, y+dy+0.5f)`. Режим `Clamp` автоматически дублирует граничные значения — дополнительных `min/max` не требуется. Яркость каждого сэмпла вычисляется по Rec.601: $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$. Свертка с ядрами Собеля выполняется с использованием масок `Gx`, `Gy`, размещённых в `__constant__` памяти видеокарты.

После завершения вычислений `d_out` копируется на хост в исходный вектор, и результат сохраняется в файл того же бинарного формата. Все выделенные ресурсы освобождаются: уничтожается `texture object`, освобождается `cudaArray` и выходной буфер. Для всех CUDA-вызовов используется макрос `CSC` с проверкой ошибок.

Описание программы

Программа состоит из функции `main`, одного CUDA-ядра и двух вспомогательных функций для чтения/записи бинарного изображения. Для удобства работы с пикселями используется 4-байтовая структура `Pixel`, где каждый канал — однобайтовое беззнаковое целое.

Ядро.

```
// Kernel function
__global__ void kernel(cudaTextureObject_t texIn, Pixel* d_out, int32_t w, int32_t h)
{
    const int xStart = blockIdx.x * blockDim.x + threadIdx.x;
    const int yStart = blockIdx.y * blockDim.y + threadIdx.y;
    const int xStep = blockDim.x * gridDim.x;
    const int yStep = blockDim.y * gridDim.y;

    for (int y = yStart; y < h; y += yStep) {
        for (int x = xStart; x < w; x += xStep) {
            float gx = 0.f;
            float gy = 0.f;

            for (int dy = -1; dy <= 1; ++dy) {
                for (int dx = -1; dx <= 1; ++dx) {
                    float fx = (float)(x + dx) + 0.5f;
                    float fy = (float)(y + dy) + 0.5f;

                    uchar4 c = tex2D<uchar4>(texIn, fx, fy);

                    // Rec.601 яркость
                    float Y = 0.299f * (float)c.x + 0.587f * (float)c.y + 0.114f * (float)c.z;

                    int8_t kx = Gx[dy + 1][dx + 1];
                    int8_t ky = Gy[dy + 1][dx + 1];
                    gx += kx * Y;
                    gy += ky * Y;
                }
            }

            int val = (int)sqrtf(gx * gx + gy * gy);
            if (val > 255) val = 255;

            size_t i = (size_t)y * (size_t)w + (size_t)x;
            d_out[i] = Pixel{ (uint8_t)val, (uint8_t)val, (uint8_t)val, 0u };
        }
    }
}
```

Ядро принимает объект текстуры `cudaTextureObject_t texIn`, указатель на выходной буфер `Pixel* d_out` и размеры `w`, `h`. Для каждого выходного пикселя собирается окно 3×3 , но соседние значения читаются из текстурной памяти вызовом `tex2D<uchar4>(texIn, x+dx+0.5f, y+dy+0.5f)`. Режим адресации `cudaAddressModeClamp` аппаратно «зажимает» координаты на границе (дублирует граничные пиксели), что избавляет от ручных `min/max`. Цвет переводится в яркость по Rec.601: $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$. Затем выполняется свёртка Собеля по маскам `Gx` и `Gy`, размещённым в `__constant__` памяти; модуль градиента $\sqrt{gx^2 + gy^2}$ насыщается до диапазона $[0, 255]$ и записывается как оттенок серого ($r=g=b=val$, $a=0$) в `d_out`.

Константные данные.

```
// Sobel kernels
__constant__ int8_t Gx[3][3] = { {-1, 0, 1}, {-2, 0, 2}, {-1, 0, 1} };
__constant__ int8_t Gy[3][3] = { {-1, -2, -1}, { 0, 0, 0}, { 1, 2, 1} };
```

Два массива Gx[3][3] и Gy[3][3] объявлены с модификатором `__constant__`, чтобы храниться в константной памяти устройства и быстро кешироваться всеми потоками.

Проверка ошибок.

```
// Check CUDA call macros
#define CSC(call) \
do { \
    cudaError_t res = call; \
    if (res != cudaSuccess) { \
        fprintf(stderr, "ERROR in %s:%d. Message: %s\n", \
            __FILE__, __LINE__, cudaGetErrorString(res)); \
        exit(0); \
    } \
} while(0)
```

Для всех CUDA-вызовов используется макрос `CSC(...)`, который проверяет код возврата, а при ошибке выводит файл/строку и текст сообщения драйвера.

Функция main.

Сначала из входного файла считываются `w`, `h` и массив `std::vector<Pixel>`, проводится проверка ограничений формата. Далее создаётся `cudaArray` под изображение и выполняется копирование данных на устройство (`cudaMemcpy2DToArray`). На основе массива формируется **texture object** с параметрами: `addressMode[0|1]=Clamp`, `filterMode=Point`, `readMode=ElementType`, `normalizedCoords=0`. Выделяется линейный выходной буфер `d_out`. Ядро запускается двумерной конфигурацией `grid(64,64)` и `block(32,8)`; каждый поток стартует из `(xStart,yStart)` и обходит кадр с шагами `xStep = blockDim.x*gridDim.x`, `yStep = blockDim.y*gridDim.y`, что корректно масштабируется на любые размеры `w×h`. После завершения вычислений результат копируется на хост и записывается тем же бинарным форматом в выходной файл. В конце освобождаются все ресурсы: уничтожается `texture object`, освобождается `cudaArray` и выходной буфер.

```

int main() {
    std::string inFilePath, outFilePath;
    std::cin >> inFilePath >> outFilePath;

    int32_t w, h;
    std::vector<Pixel> pixels;
    if (!readImage(inFilePath, w, h, pixels)) {
        return 0;
    }

    // dumpImageHex(w, h, pixels, std::cout);

    cudaArray* arr = nullptr;
    cudaChannelFormatDesc ch = cudaCreateChannelDesc<uchar4>();
    CSC(cudaMallocArray(&arr, &ch, w, h));

    CSC(cudaMemcpy2DToArray(arr, 0, 0, pixels.data(), w * sizeof(uchar4), w * sizeof(uchar4), h, cudaMemcpyHostToDevice));

    cudaResourceDesc resDesc{};
    resDesc.resType = cudaResourceTypeArray;
    resDesc.res.array.array = arr;

    cudaTextureDesc texDesc{};
    texDesc.addressMode[0] = cudaAddressModeClamp;
    texDesc.addressMode[1] = cudaAddressModeClamp;
    texDesc.filterMode = cudaFilterModePoint;
    texDesc.readMode = cudaReadModeElementType;
    texDesc.normalizedCoords = 0;

    cudaTextureObject_t texIn = 0;
    CSC(cudaCreateTextureObject(&texIn, &resDesc, &texDesc, nullptr));

    Pixel* d_out = nullptr;
    size_t bytes = w * h * sizeof(Pixel);
    CSC(cudaMalloc(&d_out, bytes));

    kernel<<<grid, block>>>(texIn, d_out, w, h);
    CSC(cudaGetLastError());
    CSC(cudaDeviceSynchronize());

    pixels.assign(w * h, {});
    CSC(cudaMemcpy(pixels.data(), d_out, bytes, cudaMemcpyDeviceToHost));

    if (!writeImage(outFilePath, w, h, pixels)) {
        cudaDestroyTextureObject(texIn);
        cudaFreeArray(arr);
        cudaFree(d_out);

        return 0;
    }

    cudaDestroyTextureObject(texIn);
    cudaFreeArray(arr);
    cudaFree(d_out);

    // dumpImageHex(w, h, pixels, std::cout);

    return 0;
}

```

Функции ввода/вывода.

```
// Helper functions (host)
bool readImage(const std::string& path, int32_t& w, int32_t& h, std::vector<Pixel>& pixels) {
    std::ifstream is(path, std::ios::binary);
    if (!is) {
        std::cerr << "ERROR: can't read input file!\n"; return false;
    }

    is.read(reinterpret_cast<char*>(&w), sizeof(int32_t));
    is.read(reinterpret_cast<char*>(&h), sizeof(int32_t));

    if (w <= 0 || h <= 0 || w >= MaxValueWH || h >= MaxValueWH) {
        std::cerr << "ERROR: width or height wrong value!\n"; return false;
    }
    // static_cast to avoid overflow
    if (static_cast<int64_t>(w) * static_cast<int64_t>(h) > MaxPixels) {
        std::cerr << "ERROR: more pixels in the image than allowed!\n"; return false;
    }

    pixels.resize(w * h);
    is.read(reinterpret_cast<char*>(pixels.data()), pixels.size() * sizeof(Pixel));

    return true;
}

bool writeImage(const std::string& path, int32_t w, int32_t h, const std::vector<Pixel>& pixels) {
    std::ofstream os(path, std::ios::binary | std::ios::trunc);
    if (!os) {
        std::cerr << "ERROR: can't open output file!\n"; return false;
    }

    os.write(reinterpret_cast<const char*>(&w), sizeof(int32_t));
    os.write(reinterpret_cast<const char*>(&h), sizeof(int32_t));
    os.write(reinterpret_cast<const char*>(pixels.data()), pixels.size() * sizeof(Pixel));

    return true;
}
```

Чтение и запись реализованы прямой работой с файлами C++ в бинарном режиме: сначала два int32 (ширина и высота), далее w*h структур Pixel. В функциях присутствуют проверки габаритов и числа пикселей на соответствие ограничениям работы. Константы размеров сетки и блока заданы в начале программы и при необходимости могут быть изменены для подбора производительности.

```
const dim3 block = dim3(32, 8);
const dim3 grid  = dim3(64, 64);
```


Для локально проверки работы программы были также реализованы функции дампа бинарных файлов в формате совпадающим с условием лабораторной работы.

```
// HEX file dump
inline void printByteHex(uint8_t b, std::ostream& os) {
    os << std::hex << std::setw(2) << std::setfill('0') << (unsigned)b;
    os << std::dec;
}

inline void printU32LE(uint32_t v, std::ostream& os) {
    printByteHex((uint8_t)(v & 0xFF), os);
    printByteHex((uint8_t)((v >> 8) & 0xFF), os);
    printByteHex((uint8_t)((v >> 16) & 0xFF), os);
    printByteHex((uint8_t)((v >> 24) & 0xFF), os);
}

void dumpImageHex(int32_t w, int32_t h, const std::vector<Pixel>& pixels, std::ostream& os) {
    printU32LE((uint32_t)w, os); os << ' ';
    printU32LE((uint32_t)h, os); os << '\n';

    for (int32_t y = 0; y < h; ++y) {
        for (int32_t x = 0; x < w; ++x) {
            const Pixel& p = pixels[(size_t)y * (size_t)w + (size_t)x];
            printByteHex(p.r, os);
            printByteHex(p.g, os);
            printByteHex(p.b, os);
            printByteHex(p.a, os);
            if (x + 1 < w) os << ' ';
        }
        os << '\n';
    }
}
```

Результаты

Для измерения производительности было использовано изображение 1200x980

Конфигурация	Время, ms
<<<dim3(8,8), dim3(16,16)>>>	4.069
<<<dim3(16,16), dim3(32,32)>>>	3.105
<<<dim3(64,64), dim3(256,256)>>>	2.012
<<<dim3(1,64), dim3(1,256)>>>	13.938
<<<dim3(1024,1), dim3(1024,1)>>>	39.824

CPU time = 381.312 ms

Примеры обработанных изображений





Выводы

В результате данной работы я научился обрабатывать изображения с помощью GPU, а также работать с двумерной сеткой потоков и текстурной памятью. Получилось реализовать выделение контуров на изображении с помощью метода Собеля, а также вспомнить как программировать на C++.

Из типовых задач, решаемых методом Собеля, можно выделить предобработку изображений перед их дальнейшей сегментацией, а также для детекции изменений (например, в видеонаблюдении) - сравнение карт градиента между кадрами для поиска новых/исчезнувших контуров. Применение в компьютерном зрении, робототехнике для четкого определения границ.

Если говорить честно, то применение моей реализации не вижу ни в какой из областей – это учебный «проект», который явно написан неоптимально и может быть сделан гораздо лучше. Единственно, что приходит на ум – поиграться с выделением контуров на различных изображениях, возможно получится что-нибудь забавное, как в примерах обработанных изображений.

Проведенные замеры демонстрируют значительное преимущество GPU-реализации над CPU-версией алгоритма. Наилучшие результаты показывают

конфигурации с двумерной организацией потоков: $\langle\langle\langle\text{dim3}(64,64), \text{dim3}(256,256)\rangle\rangle\rangle$ и $\langle\langle\langle\text{dim3}(16,16), \text{dim3}(32,32)\rangle\rangle\rangle$ с временем выполнения 2.012 мс и 3.105 мс соответственно.

Одномерные конфигурации потоков демонстрируют существенно худшие результаты. Например, $\langle\langle\langle\text{dim3}(1024,1), \text{dim3}(1024,1)\rangle\rangle\rangle$ показывает время 39.824 мс на крупном изображении. Это объясняется не эффективным использованием текстурной памяти и кэша при линейной организации доступа к пикселям.